Project Report for
Chulalongkorn University's Academic Year 2025, Semester 2 offering of
2110215 PROG METH I

AnimalWizard

6833058921 Tapanan Rojkajornnapalai
6833128121 Thianrawit Phansuwannakit
6833096721 Tinnaphop Choonchuachan
6833233321 Ratchaphon Pungtamgerdpol

Table of Contents

Introduction **2**

Basic Gameplay **3**

Character Select 3

In a Match 3

Attacks & Abilities 3

End Screen 4

JavaDoc **5**

UML Diagrams **6**

Package attack 6

Package character 7

Package game 8

Package renderer 9

Introduction

AnimalWizard is a two-player, turn-based 2D fighting game built with JavaFX. Players choose an animal character — Dog or Cat — and take turns aiming and firing elemental projectiles at each other. The first player to reduce the opponent's HP to zero wins.

The project is structured across five packages: `application`, `attack`, `character`, `game`, and `renderer`.

Characters

Character	Max HP	Special Ability
Dog	10 HP	Heal — restores 1 HP and ends the turn. Cooldown: 4 turns.
Cat	10 HP	Perfect Aim — locks trajectory to hit the opponent exactly; cancels wind on self. Cooldown: 4 turns.

Attacks

Attack	Effect	Direct Damage	Cooldown
Normal	Plain projectile, no status	1 HP	None
Fire	Burns target for 2 ticks (~1 s each)	2 HP + 2 burn ticks	3 turns
Ice	Freezes target; next turn skipped automatically	1 HP + freeze	2 turns
Wind	Pushes target's sprite horizontally for 1 turn	1 HP + wind push	2 turns

Basic Gameplay

Character Select

Each player clicks a character card (Dog or Cat) to make their selection. The FIGHT! button enables once both players have chosen, then starts the match.

In a Match

The game is turn-based. Each turn: (1) click a skill button to select an attack or ability, (2) press **SPACE** to start aiming, (3) press **SPACE** again to fire.

Input	Action
Click skill circle (NRM / FIRE / ICE / WIND)	Select attack type for this turn
Click ability circle (rightmost)	Activate special ability (Heal or Perfect Aim)
SPACE — 1st press	Start aiming; arrow oscillates
SPACE — 2nd press	Fire projectile at current angle

The HUD shows HP bars, player names, VS, and a turn indicator. When frozen, the turn is skipped automatically after 2 seconds. Cooldown buttons display a countdown number.

Attacks & Abilities

- **Normal** — 1 HP on hit.
- **Fire** — 2 HP on hit + burns for 2 ticks (~1 s each).
- **Ice** — 1 HP on hit + freezes target (skips their next turn).
- **Wind** — 1 HP on hit + pushes target's sprite horizontally for 1 turn.
- **Dog Heal** — restores 1 HP and ends the turn. Cooldown 4 turns.
- **Cat Perfect Aim** — cancels wind on self and locks trajectory to hit the opponent exactly. Cooldown 4 turns.

A miss (projectile leaves the screen without hitting the hitbox) loses the turn: *MISSED! Turn lost.*

End Screen

When a player's HP reaches zero the end screen shows the winner. Options: **Rematch** (same characters, full HP), **Change Characters** (back to selection), **Main Menu**.

JavaDoc

The methods of each class and their purpose can be found in the documentation, which is at

file:///c:/Programming/visual%20studio%20code/Progmeth/ProgmethFinalProject/docs/index.html

Package	Classes / Interfaces
`application`	Main, HandleInput
`application.scene`	StartScene, CharacterSelectScene, GamePlayScene, EndScene
`attack`	Attackable (interface), NormalAttack, FireAttack, IceAttack, WindAttack
`character`	AbilityType (enum), BasePlayer, DogPlayer, CatPlayer
`game`	Controller, Projectile, PhysicsUtils, StatusManager, SoundManager
`renderer`	GameRenderer, HudRenderer, ProjectileRenderer, EffectRenderer, SkillButtonRenderer

UML Diagrams

UML Class Diagrams split by package. Each diagram occupies one page.

Package attack

Defines the `Attackable` interface and four concrete classes (NormalAttack, FireAttack, IceAttack, WindAttack). Each provides `attack()`, `getName()`, and `getCooldown()`.

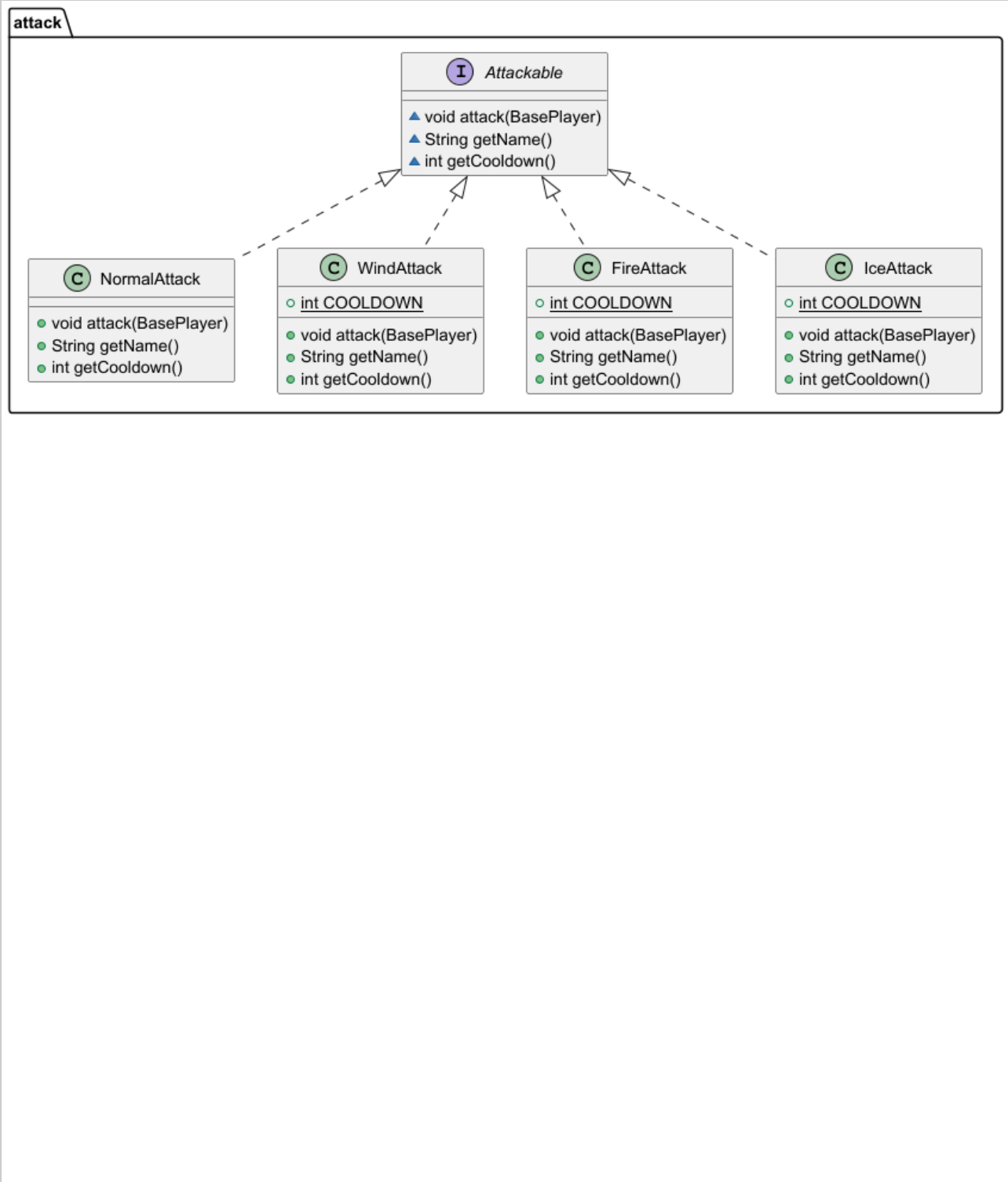
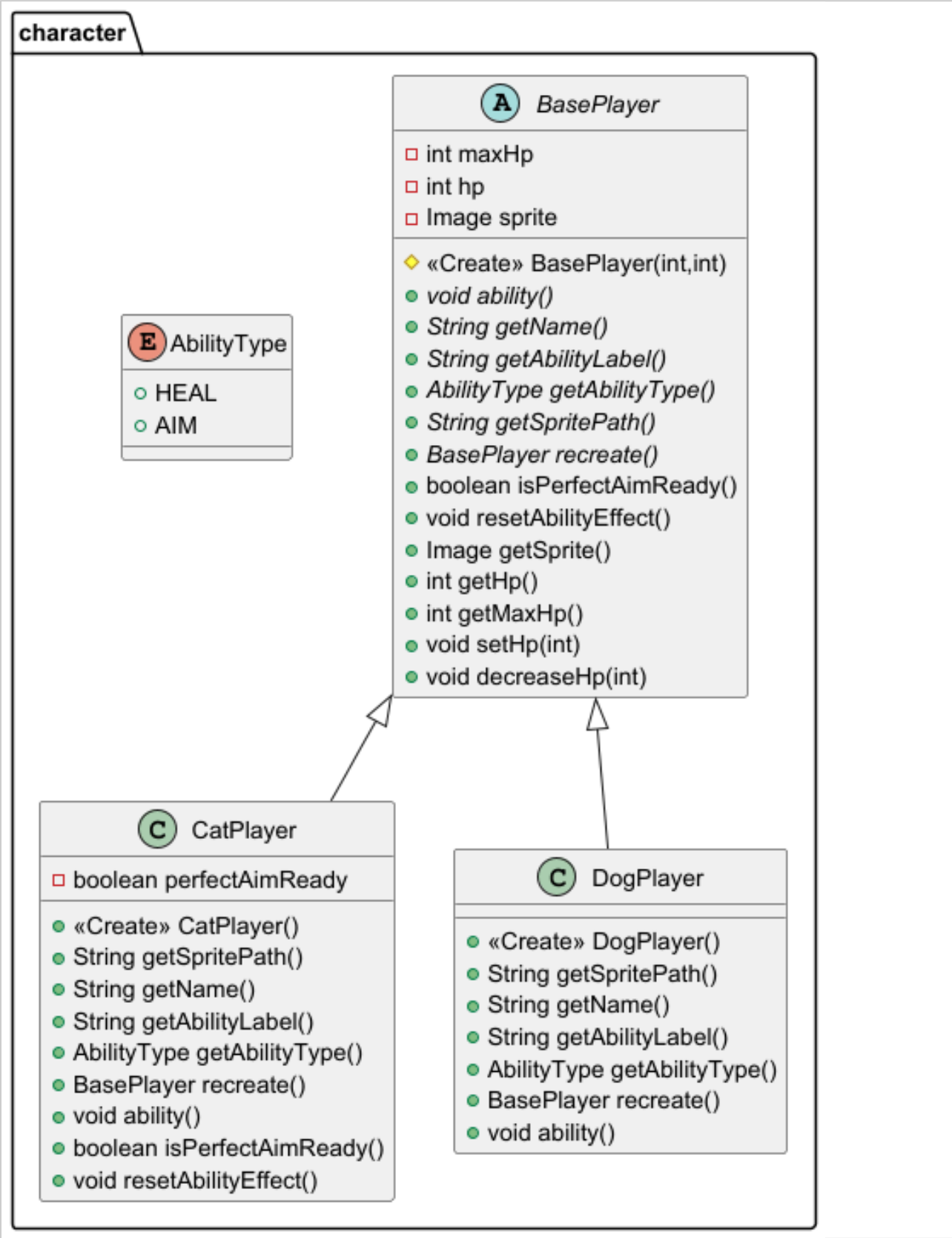


Figure 1: Package attack

Package character

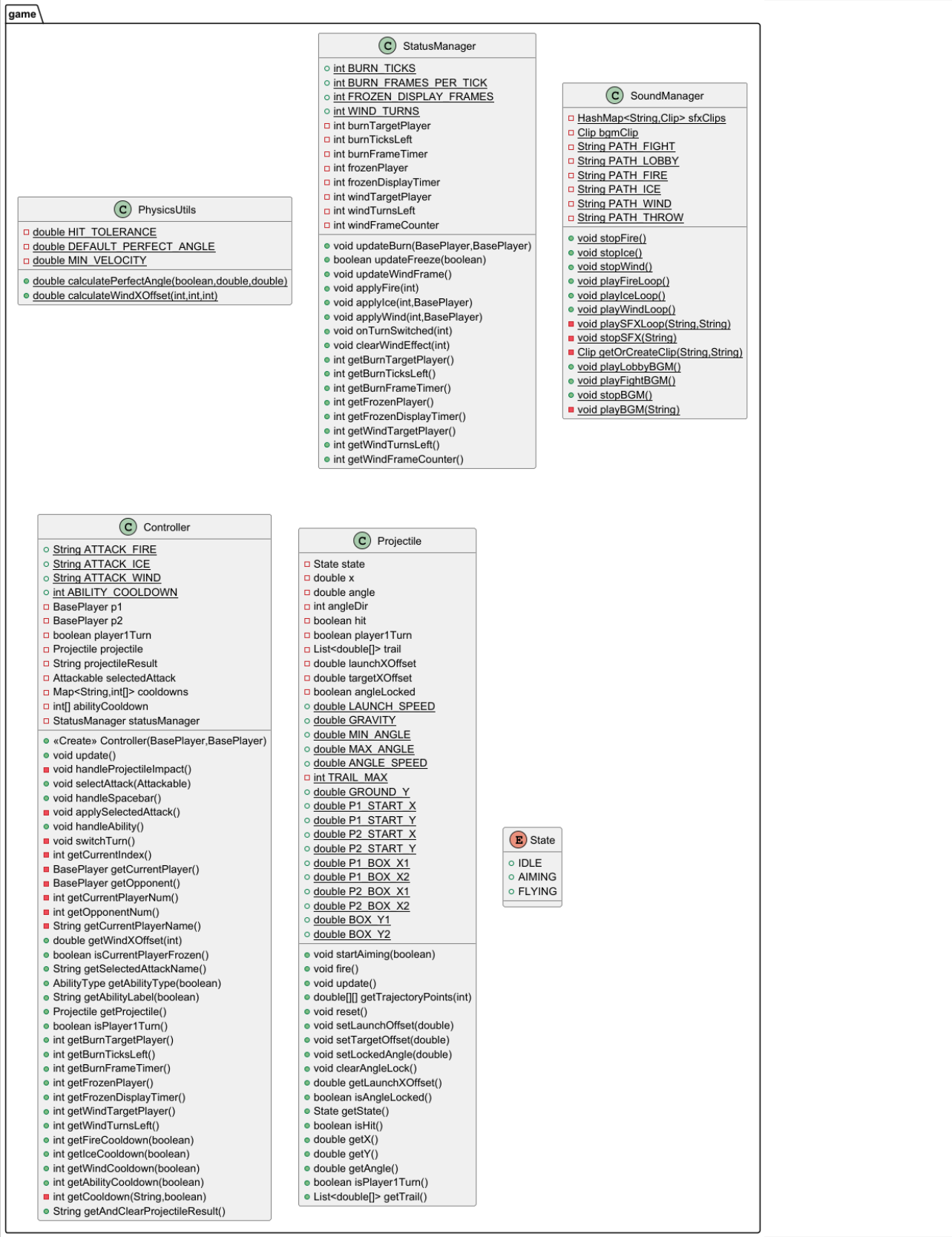
Abstract `BasePlayer` with two concrete subclasses `DogPlayer` and `CatPlayer`. Both extend `BasePlayer` and implement `ability()`, `getAbilityType()`, `getAbilityLabel()`, and `recreate()`. The `AbilityType` enum distinguishes HEAL from AIM.



Package game

Figure 2: Package character

`Controller` manages turn order and cooldowns, delegating to `StatusManager` (Burn/Freeze/Wind) and `PhysicsUtils` (physics). `Projectile` models the IDLE→AIMING→FLYING lifecycle. `SoundManager` handles all BGM and SFX.



Package renderer

Figure 3: Package game

`GameRenderer` orchestrates four sub-renderers: `HudRenderer` (HP bars, turn indicator), `SkillButtonRenderer` (skill circles with cooldown overlay), `ProjectileRenderer` (aim arrow and trail), and `EffectRenderer` (Burn/Freeze/Wind particles).

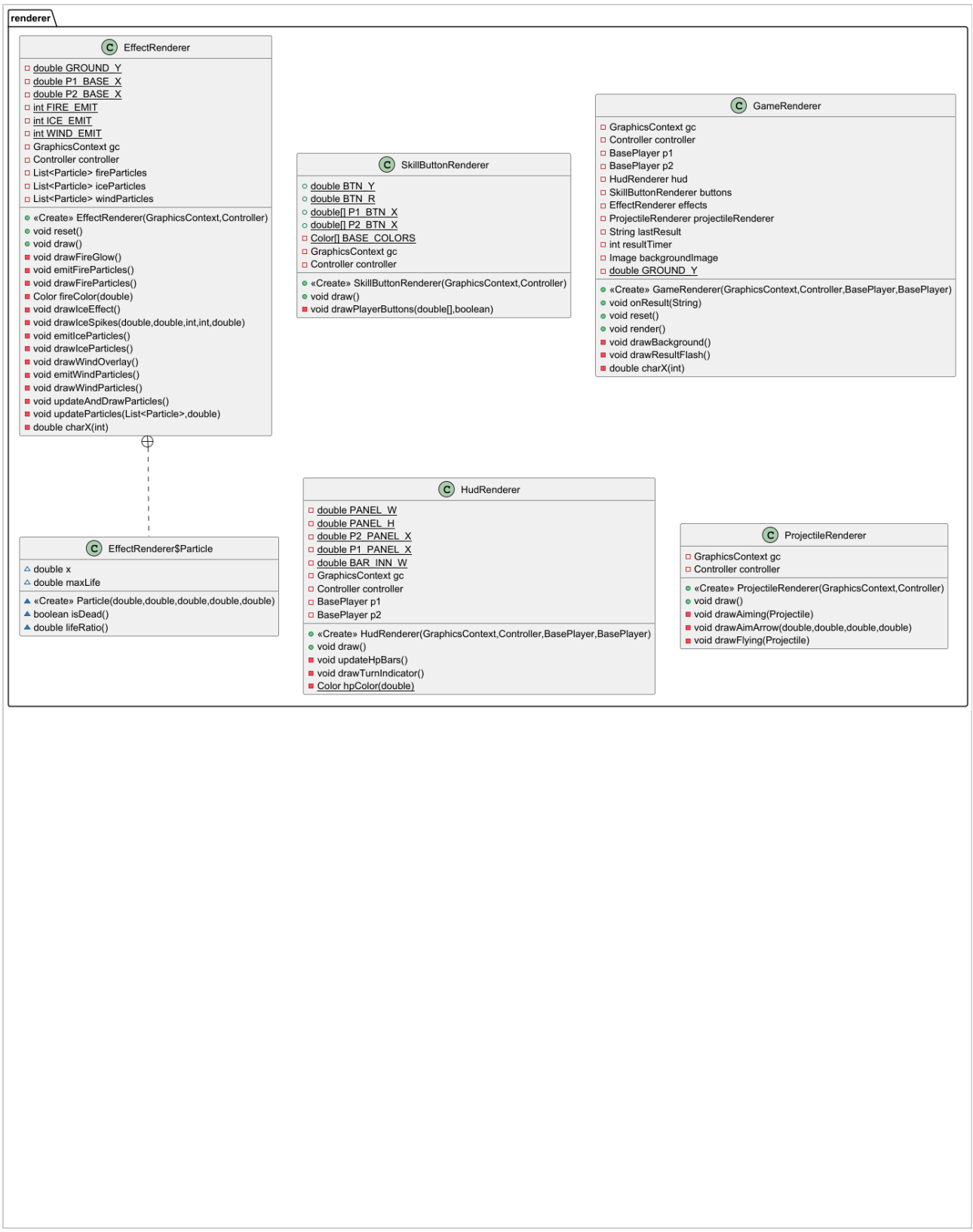


Figure 4: Package renderer